真实接口联调执行指南

> 生成日期：2026-03-17 | 适用项目：轻院二手交易平台

⚠️ 第一步：环境准备（必须先执行）

1.0 前置检查（建议先执行）

1.0.1 工具版本检查

```
# 检查Node.js版本（需 ≥ 16）
node -v

# 检查npm版本
npm -v

# 检查Maven版本（需 ≥ 3.6）
mvn -v

# 检查Java版本（需 ≥ 17）
java -version
```

1.0.2 端口占用检查

```
# Windows - 检查端口占用情况
netstat -ano | findstr :3306   # MySQL
netstat -ano | findstr :6379   # Redis
netstat -ano | findstr :8080   # 后端服务

# 如果端口被占用，记录PID并停止进程
taskkill /PID <进程ID> /F
```

```
# macOS/Linux - 检查端口占用情况
lsof -i :3306   # MySQL
lsof -i :6379   # Redis
lsof -i :8080   # 后端服务
```

1.0.3 服务状态检查

```
# Windows - 检查服务状态
# 方法1：服务管理器
services.msc

# 方法2：命令行
net start | findstr mysql
net start | findstr redis
```

```
# macOS - 检查服务状态
brew services list | grep mysql
brew services list | grep redis
```

1.1 修改小程序配置

```
// miniapp/utils/request.js 第5行
const USE_MOCK = false  // 关闭Mock，使用真实接口
```

1.2 启动基础服务

Windows 系统启动命令

1. 启动MySQL服务

方法一：右键"此电脑" → 管理 → 服务 → 找到MySQL → 启动

方法二：命令行

net start mysql

2. 启动Redis服务

方法一：命令行

redis-server --service-start

或指定端口

redis-server --port 6379

方法二：直接运行redis-server（保持窗口打开）

redis-server

```
```powershell
# 方式2：使用XAMPP/WAMP等集成环境

# 如果使用XAMPP：
# 1. 打开XAMPP Control Panel
# 2. 启动MySQL
# 3. 启动Redis（如果集成了Redis）
```
```

macOS 系统启动命令

```
# 1. 启动MySQL服务
# 方式一：使用homebrew
brew services start mysql

# 方式二：直接运行
mysql.server start

# 2. 启动Redis服务
brew services start redis

# 或直接运行
redis-server
```

Linux 系统启动命令

```
# 1. 启动MySQL服务
sudo systemctl start mysql
# 或
sudo systemctl start mysqld

# 2. 启动Redis服务
sudo systemctl start redis
# 或
sudo redis-server
```

1.3 启动后端服务

```
# 1. 进入项目目录
cd G:\Code\Graduation_project

# 2. 启动后端服务（端口8080）
mvn spring-boot:run

# 或先编译再运行
mvn clean compile
mvn spring-boot:run
```

后端启动成功标志：

```
Started SecondhandApplication in X.XXX seconds
```

1.4 启动小程序端

```
# 1. 进入小程序目录
cd G:\Code\Graduation_project\miniapp

# 2. 安装依赖（首次运行）
npm install

# 3. 启动开发服务器
npm run dev
# 或
npm run serve
```

1.5 启动管理端

```
# 1. 进入管理端目录
cd G:\Code\Graduation_project\admin

# 2. 安装依赖（首次运行）
npm install

# 3. 启动开发服务器
npm run dev

# 4. 浏览器访问 http://localhost:5173
```

1.6 微信开发者工具配置

1. 打开微信开发者工具
2. 进入 → 项目详情 → 本地设置
3. 勾选 ✅ **不校验合法域名、web-view（业务域名）、TLS版本以及HTTPS证书**
4. 刷新项目

⚙️ 第二步：多账号测试准备（IM和订单模块必需）

2.1 测试账号准备

IM即时通讯和订单模块需要**两个微信号同时在线**才能完整测试。建议准备以下测试账号：

角色	账号类型	准备方式	用途
买家账号（A）	微信号1	微信登录小程序	发起购买、发送消息
卖家账号（B）	微信号2	微信登录小程序	发布商品、接收消息

2.2 账号注册流程

```
# 账号A注册（买家）
1. 打开微信开发者工具
2. 点击"微信一键登录"
3. 授权后自动创建账号
4. 记录用户ID（数据库user表）

# 账号B注册（卖家）
1. 退出账号A或使用另一个微信开发者工具实例
2. 点击"微信一键登录"
3. 授权后自动创建账号
4. 记录用户ID（数据库user表）
```

2.3 测试数据准备

```
-- 在Navicat中执行，为账号B准备测试商品

-- 1. 先查询账号B的用户ID
SELECT id, nick_name FROM user WHERE id = [账号B的ID];

-- 2. 为账号B插入测试商品（状态为1-在售）
INSERT INTO product (user_id, title, price, description, category_id, campus_id,
status, create_time, update_time)
VALUES (账号B的ID, '测试商品', 99.99, '这是一条测试商品描述', 1, 1, 1, NOW(), NOW());
```

🗄️ 第三步：数据库初始化说明

3.1 创建数据库

```
-- 登录MySQL
mysql -u root -p

-- 创建数据库
CREATE DATABASE secondhand CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

3.2 初始化数据库表结构

```
# 进入项目目录
cd G:\Code\Graduation_project

# 执行初始化脚本
mysql -u root -p secondhand < sql/init.sql
```

3.3 初始化测试数据

```sql
-- 1. 插入默认管理员（密码：123456）
INSERT INTO employee (username, password, name, role, status, create_time,
update_time)
VALUES ('admin', '$2a$10$N.zmdr9k7uOCQb376NoUnuTJ8iAt6Z5EHsM8lE9lBOsl7iKTVKIUi',
'超级管理员', 1, 1, NOW(), NOW());

-- 2. 插入校区数据
INSERT INTO campus (name, code, sort, status, create_time, update_time) VALUES
('南海北', 'nanhai_north', 1, 1, NOW(), NOW()),
('南海南', 'nanhai_south', 2, 1, NOW(), NOW()),
('新港', 'xingang', 3, 1, NOW(), NOW());

-- 3. 插入分类数据
INSERT INTO category (name, sort, status, create_time, update_time) VALUES
('书籍', 1, 1, NOW(), NOW()),
('服饰', 2, 1, NOW(), NOW()),
('生活', 3, 1, NOW(), NOW()),
('电子设备', 4, 1, NOW(), NOW()),
('运动设备', 5, 1, NOW(), NOW()),
('潮玩娱乐', 6, 1, NOW(), NOW());

-- 4. 插入面交地点数据
INSERT INTO meeting_point (campus_id, name, sort, status, create_time,
update_time) VALUES
(1, '南海北校区-宿舍楼A', 1, 1, NOW(), NOW()),
(1, '南海北校区-图书馆', 2, 1, NOW(), NOW()),
(2, '南海南校区-食堂', 1, 1, NOW(), NOW()),
(3, '新港校区-教学楼', 1, 1, NOW(), NOW());
```

3.4 验证数据库初始化成功

```sql
-- 检查表是否创建成功
SHOW TABLES;

-- 检查数据是否插入成功
SELECT COUNT(*) AS admin_count FROM employee;
SELECT COUNT(*) AS campus_count FROM campus;
SELECT COUNT(*) AS category_count FROM category;
```

3.5 配置文件检查

```yaml
# 文件：src/main/resources/application.yml
# 重点检查项：
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/secondhand?
useSSL=false&serverTimezone=Asia/Shanghai
    username: root
    password: root1 ·11a  # 注意：密码中有空格，请确保正确
  data:
    redis:
      host: localhost
      port: 6379
```

```
// 文件：miniapp/utils/request.js
const USE_MOCK = false  // 必须是false

// 文件：miniapp/utils/constant.js
const BASE_URL = 'http://localhost:8080'  // 确保端口正确

---

## 📋 第四步：分模块联调检查

### 模块1：登录注册

| 序号 | 检查项 | 测试方法 | 预期结果 | 验证方式 |
|------|--------|----------|----------|----------|
| 1 | 微信一键登录 | 点击微信登录按钮 | 弹出授权，授权后返回Token | 查看本地存储token |
| 2 | 账号密码登录 | 输入正确账号密码 | 登录成功，返回Token | 返回token |
| 3 | 账号密码登录 | 输入错误密码 | 提示"账号或密码错误" | 提示信息 |
| 4 | 账号密码登录 | 密码错误5次 | 账号锁定15分钟 | 提示并Redis计数 |
| 5 | 短信验证码登录 | 输入手机号+验证码 | 登录成功，新手机号自动注册 | 返回token |

**接口列表**：
- `POST /mini/user/wx-login` - 微信登录（公开）
- `POST /mini/user/login` - 手机号密码登录（公开）
- `POST /mini/user/sms/send` - 发送短信验证码（公开）
- `POST /mini/user/sms-login` - 短信验证登录（公开）

**预期实现结果**：用户可以通过三种方式登录小程序，登录成功后跳转到首页。

---

### 模块2：首页

| 序号 | 检查项 | 测试方法 | 预期结果 | 验证方式 |
|------|--------|----------|----------|----------|
| 1 | 校区切换 | 切换校区 | 商品列表更新为该校区 | 页面刷新 |
| 2 | Banner运营位 | 查看Banner | 返回当前校区Banner，点击可跳转 | 接口返回 |
| 3 | 分类入口 | 点击分类 | 进入商品列表（筛选该分类） | 页面跳转 |
| 4 | 商品列表 | 首页下拉 | 分页加载商品 | 分页功能 |

**接口列表**：
- `GET /mini/campus/list` - 校区列表
- `GET /mini/banner/list?campusId=xxx` - Banner列表
```

- `GET /mini/category/list` - 分类列表
- `GET /mini/product/list` - 商品列表

预期实现结果：首页展示当前校区的Banner、分类入口和最新发布的商品流，切换校区后内容随之更新。

模块3：搜索筛选

序号	检查项	测试方法	预期结果	验证方式
1	关键词搜索	输入关键词搜索	返回匹配商品	接口返回
2	热门搜索	获取热搜词	返回热搜词列表	接口返回
3	多维度筛选	筛选分类/价格/成色	筛选生效	筛选功能
4	价格排序	选择升序/降序	排序生效	排序功能

接口列表：
- `GET /mini/product/list?keyword=xxx` - 关键词搜索
- `GET /mini/search/hot-keywords` - 热门搜索词

预期实现结果：用户可以输入关键词搜索商品，热搜词展示在搜索页面，支持多维度筛选和价格排序。

模块4：商品模块

序号	检查项	测试方法	预期结果	验证方式
1	商品详情	查看商品详情	返回完整商品信息+卖家信息	接口返回
2	商品发布	填写信息发布	进入审核队列	数据库+页面
3	图片上传	上传商品图片	返回图片URL	接口返回
4	商品下架	点击下架	商品状态变为已下架	数据库+页面
5	商品上架	点击上架	商品重新上架	数据库+页面
6	删除商品	点击删除	商品逻辑删除	数据库

接口列表：
- `GET /mini/product/detail/{id}` - 商品详情
- `POST /mini/product/publish` - 发布商品
- `POST /mini/product/update` - 编辑商品
- `POST /mini/product/off-shelf` - 下架商品 (body: `{productId: xxx}`)
- `POST /mini/product/on-shelf` - 上架商品 (body: `{productId: xxx}`)
- `POST /mini/product/delete` - 删除商品 (body: `{productId: xxx}`)

预期实现结果：用户可以发布商品（需审核），查看商品详情，编辑/下架/删除自己发布的商品。

模块5：收藏模块

序号	检查项	测试方法	预期结果	验证方式
1	收藏商品	点击收藏	收藏成功	接口返回+页面
2	取消收藏	点击取消	取消成功	接口返回+页面
3	收藏列表	查看收藏	返回收藏商品列表	接口返回
4	判断是否收藏	查看商品详情	显示是否已收藏	接口返回

接口列表：
- `POST /mini/favorite/add` - 收藏商品 (body: `{productId: xxx}`)
- `POST /mini/favorite/cancel` - 取消收藏 (body: `{productId: xxx}`)
- `GET /mini/favorite/list` - 收藏列表
- `GET /mini/favorite/check/{productId}` - 是否已收藏

预期实现结果：用户可以收藏/取消收藏商品，收藏列表展示所有已收藏的商品。

模块6：IM即时通讯（重点）

序号	检查项	测试方法	预期结果	验证方式
1	WebSocket连接	打开聊天页面	ws://localhost:8080/ws/chat?token=xxx	控制台网络
2	文字消息发送	发送消息	对方实时收到	对方页面显示
3	商品卡片	点击"我想要"	自动发送商品卡片	IM消息
4	订单卡片	创建订单	自动发送订单卡片	IM消息
5	会话列表	查看聊天列表	显示所有会话	接口返回
6	未读数管理	收到消息	未读数正确累加	页面显示
7	标记已读	点击会话	未读数清零	页面更新

WebSocket地址：`ws://localhost:8080/ws/chat?token={jwt_token}`

消息协议：
```json
// 发送文本消息
{"type": "CHAT", "data": {"receiverId": xxx, "productId": xxx, "msgType": 1, "content": "xxx"}}
```

```
// 发送商品卡片
{"type": "CHAT", "data": {"receiverId": xxx, "productId": xxx, "msgType": 2}}

// 发送订单卡片
{"type": "CHAT", "data": {"receiverId": xxx, "orderId": xxx, "msgType": 3}}

// 心跳
{"type": "PING"}
```

REST接口：

- `GET /mini/chat/sessions` - 会话列表
- `POST /mini/chat/session/create` - 创建会话
- `POST /mini/chat/session/delete` - 删除会话
- `POST /mini/chat/session/top` - 置顶/取消置顶会话
- `GET /mini/chat/messages?sessionKey=xxx` - 消息历史
- `POST /mini/chat/read` - 标记已读 (body: `{sessionKey: xxx}`)
- `GET /mini/chat/unread-total` - 未读消息总数

预期实现结果：用户可以通过WebSocket实时发送文字消息、商品卡片、订单卡片，会话列表展示所有聊天记录，未读数实时更新。

模块7：订单模块

序号	检查项	测试方法	预期结果	验证方式
1	创建订单	点击"确认购买"	创建订单成功，发送订单卡片给卖家	数据库+IM
2	订单列表(买家)	查看购买订单	显示我购买的订单	页面展示
3	订单列表(卖家)	查看卖出订单	显示我卖出的订单	页面展示
4	确认收货	点击确认收货	订单状态变为已完成	数据库+页面
5	取消订单	点击取消	取消成功，通知对方	数据库+通知
6	超时自动取消	等待72小时	订单自动取消	定时任务+通知

接口列表：

- `POST /mini/order/create` - 创建订单 (body: `{productId: xxx}`)
- `GET /mini/order/list?role=buyer&status=xxx` - 订单列表
- `GET /mini/order/detail/{id}` - 订单详情

- `POST /mini/order/confirm` - 确认收货 (body: `{orderId: xxx}`)
- `POST /mini/order/cancel` - 取消订单 (body: `{orderId: xxx, cancelReason: xxx}`)
- `POST /mini/order/delete` - 删除订单 (body: `{orderId: xxx}`)

预期实现结果：买卖双方可以创建订单、管理订单状态，订单超时自动取消，确认收货后进入评价流程。

模块8：评价模块

序号	检查项	测试方法	预期结果	验证方式
1	提交评价	交易完成后评价	评价成功提交	接口返回
2	三维度评分	选择描述/态度/体验	评分正确保存	数据库
3	查看评价	查看订单评价	评价页面展示	页面展示
4	自动好评	7天未评价	系统自动默认好评	数据库+页面

接口列表：

- `POST /mini/review/submit` - 提交评价
- `GET /mini/review/list?orderId=xxx` - 评价列表

预期实现结果：交易完成后双方可以互评，7天未评价自动生成默认好评，卖家综合评分实时更新。

模块9：消息中心

序号	检查项	测试方法	预期结果	验证方式
1	通知列表	查看消息中心	返回通知列表	接口返回
2	分类筛选	筛选交易/系统	按分类筛选	筛选功能
3	标记已读	点击消息	单条已读	页面更新
4	全部已读	点击全部已读	全部标记已读	页面更新
5	未读数统计	查看未读数	返回未读数	接口返回

接口列表：

- `GET /mini/notification/list` - 通知列表
- `POST /mini/notification/read` - 标记单条已读 (body: `{id: xxx}`)
- `POST /mini/notification/read-all` - 全部已读
- `GET /mini/notification/unread-count` - 未读数统计

预期实现结果：用户可以在消息中心查看所有交易通知和系统通知，支持分类筛选和标记已读。

模块10：管理端

序号	检查项	测试方法	预期结果	验证方式
1	管理员登录	输入账号密码	登录成功，返回Token	页面跳转
2	商品审核	审核商品	通过/驳回生效，通知用户	数据库+通知
3	认证审核	审核认证	通过/驳回生效	数据库+通知
4	数据统计	查看统计	显示关键指标	页面展示
5	举报处理	处理举报	处理/驳回举报	数据库+通知
6	用户管理	封禁/解封用户	用户状态变更	数据库+通知
7	订单管理	查看订单	订单列表展示	页面展示

管理端详细接口：

```
# 管理员
POST /admin/employee/login - 管理员登录
GET /admin/employee/info - 获取管理员信息
GET /admin/employee/page - 管理员列表

# 商品审核
GET /admin/product/page - 商品列表(分页)
GET /admin/product/detail/{id} - 商品详情
POST /admin/product/approve - 审核通过
POST /admin/product/reject - 审核驳回
POST /admin/product/batch-approve - 批量通过
POST /admin/product/force-off - 强制下架

# 认证审核
GET /admin/auth/page - 认证列表
GET /admin/auth/detail/{id} - 认证详情
POST /admin/auth/approve - 认证通过
POST /admin/auth/reject - 认证驳回

# 举报处理
GET /admin/report/page - 举报列表
GET /admin/report/detail/{id} - 举报详情
POST /admin/report/handle - 处理举报

# 数据统计
GET /admin/stats/overview - 数据概览
GET /admin/stats/trend - 趋势数据
GET /admin/stats/campus - 校区统计
GET /admin/stats/category - 分类统计

# 用户管理
GET /admin/user/page - 用户列表
POST /admin/user/ban - 封禁用户
POST /admin/user/unban - 解封用户

# 订单管理
GET /admin/order/page - 订单列表
GET /admin/order/detail/{id} - 订单详情

# Banner管理
GET /admin/banner/page - Banner列表
POST /admin/banner/add - 添加Banner
```

```
POST /admin/banner/update - 更新Banner
POST /admin/banner/delete - 删除Banner

# 分类管理
GET /admin/category/page - 分类列表
GET /admin/category/list - 分类下拉列表
POST /admin/category/add - 添加分类
POST /admin/category/update - 更新分类
POST /admin/category/delete - 删除分类

# 校区管理
GET /admin/campus/list - 校区列表
POST /admin/campus/add - 添加校区
POST /admin/campus/update - 更新校区
GET /admin/campus/meeting-point/list/{campusId} - 面交地点
POST /admin/campus/meeting-point/add - 添加面交地点
POST /admin/campus/meeting-point/update - 更新面交地点
POST /admin/campus/meeting-point/delete - 删除面交地点

# 公告管理
GET /admin/notice/page - 公告列表
POST /admin/notice/add - 添加公告
POST /admin/notice/update - 更新公告
POST /admin/notice/delete - 删除公告

# 学院管理
GET /admin/college/list - 学院列表
POST /admin/college/add - 添加学院
POST /admin/college/update - 更新学院
POST /admin/college/delete - 删除学院
```

管理端地址：`http://localhost:5173`

默认管理员账号：

- 用户名：admin
- 密码：123456

预期实现结果：管理员可以登录后台，审核商品和校园认证，查看数据统计，管理Banner、分类、校区等信息。

🔧 第五步：网络异常场景测试

6.1 网络断开重连

序号	测试场景	测试方法	预期结果
1	移动网络切换	切换WiFi/移动数据	接口自动重连，数据正常
2	飞行模式	开启/关闭飞行模式	恢复后自动重连
3	后端重启	重启后端服务	小程序显示连接断开，自动重连

6.2 WebSocket重连机制

```
// WebSocket断开后重连逻辑（小程序端）
let reconnectCount = 0;
const maxReconnectCount = 5;

function onSocketClose() {
  if (reconnectCount < maxReconnectCount) {
    setTimeout(() => {
      connectWebSocket();
      reconnectCount++;
    }, 3000); // 3秒后重连
  }
}
```

6.3 Token过期处理

序号	测试场景	测试方法	预期结果
1	Token过期	等待24小时后操作	提示重新登录
2	Token无效	修改Token为错误值	返回401，跳转登录页

测试方法:

```
# 1. 登录获取Token
# 2. 在Redis中手动删除Token
redis-cli DEL "token:user:xxx"
# 3. 尝试发送请求
# 4. 预期：返回401，提示重新登录
```

⚡ 第六步：性能与并发测试

7.1 并发下单防护测试

```
# 测试场景：同一商品同时被多个买家下单

# 1. 准备测试商品（状态为在售）
# 2. 使用Postman模拟并发请求
# 3. 预期：只有第一个请求成功，其他请求返回"该商品已有待处理订单"

# 测试命令（使用JMeter或curl）
curl -X POST http://localhost:8080/mini/order/create \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer {token}" \
  -d '{"productId": 1}'
```

7.2 压力测试

测试项	测试方法	预期结果	性能指标
商品列表查询	100个并发请求	正常返回	响应时间 < 500ms
搜索接口	50个并发请求	正常返回	响应时间 < 800ms
WebSocket并发	50个同时连接	稳定运行	无断开
订单创建	10个并发请求	仅1个成功	其他返回友好提示

7.3 性能监控

```
# 查看后端性能（JMeter或wrk）
# 关注指标：
# - QPS（每秒请求数）
# - 响应时间（P50/P95/P99）
# - 错误率
# - 资源使用率（CPU/内存）
```

🚀 第七步：生产环境部署验证

8.1 生产环境切换检查项

序号	检查项	说明
1	修改数据库配置	application.yml 中的数据库地址、用户名、密码
2	修改Redis配置	生产环境Redis地址和密码
3	修改JWT密钥	使用强密钥（至少32位随机字符串）
4	微信小程序配置	修改业务域名
5	文件上传配置	切换到OSS/CDN存储
6	USE_MOCK配置	设置为 `false`

8.2 Nginx反向代理配置（可选）

```
# nginx.conf 配置示例

http {
    upstream backend {
        server 127.0.0.1:8080;
    }

    server {
        listen 80;
        server_name your-domain.com;

        # 静态资源
        location /uploads/ {
            alias /var/www/html/uploads/;
        }

        # API反向代理
        location / {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        # WebSocket反向代理
        location /ws/ {
            proxy_pass http://backend;
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection "upgrade";
            proxy_read_timeout 86400;
        }
    }
}
```

8.3 部署后验证流程

```
# 1. API接口验证
curl http://your-domain.com/mini/campus/list

# 2. WebSocket连接验证
wscat -c ws://your-domain.com/ws/chat?token=xxx

# 3. 文件上传验证
# 上传图片后访问 http://your-domain.com/uploads/xxx.jpg

# 4. 微信开发者工具验证
# 1. 取消勾选"不校验合法域名"
# 2. 配置业务域名
# 3. 验证所有功能正常
```

8.4 常见部署问题

问题	可能原因	解决方案
接口404	域名未解析/Nginx配置错误	检查DNS和Nginx配置
WebSocket连接失败	端口未开放/防火墙	检查安全组和防火墙
图片不显示	OSS配置错误/权限不足	检查OSS Bucket权限
微信登录失败	AppID/AppSecret错误	检查微信配置

🔧 第八步：问题排查

8.1 常见问题速查

问题现象	可能原因	解决方案
请求404	接口路径错误/后端未启动	检查后端服务+接口路径
请求401	Token失效/未携带	重新登录获取Token
请求500	后端异常	查看后端控制台日志
图片不显示	文件路径错误/未配置静态资源	检查upload配置
WebSocket连接失败	端口不通/Token错误	检查WS地址+Token
数据库操作失败	数据库未创建/连接失败	检查MySQL配置

8.2 详细问题排查

问题1：后端启动失败

```
# 现象：mvn spring-boot:run 报错

# 排查步骤：
# 1. 检查MySQL是否启动
net start | findstr mysql

# 2. 检查Redis是否启动
net start | findstr redis

# 3. 检查数据库是否存在
mysql -u root -p -e "SHOW DATABASES;"

# 4. 检查端口占用
netstat -ano | findstr :8080

# 5. 查看详细错误日志
mvn spring-boot:run 2>&1 | tee backend.log
```

问题2：小程序请求失败

```
// 现象：小程序提示"网络请求失败"

// 排查步骤（微信开发者工具）：
# 1. 打开控制台 → Network面板
# 2. 查看请求状态码
# 3. 查看请求参数和响应

// 常见错误：
// - 404：检查BASE_URL是否正确
// - 401：检查Token是否有效
// - 500：检查后端日志
```

问题3：WebSocket连接失败

```
// 现象：WebSocket连接失败

// 排查步骤：
# 1. 检查后端WebSocket配置
# 文件：src/main/java/com/qingyuan/secondhand/config/WebSocketConfig.java

# 2. 检查Token是否有效
# 在微信开发者工具Console执行：
const token = uni.getStorageSync('token')
console.log('Token:', token)

# 3. 检查WebSocket地址
# 正确格式：ws://localhost:8080/ws/chat?token=xxx

# 4. 使用第三方工具测试
# Postman WebSocket / WebSocket King Client
ws://localhost:8080/ws/chat?token=xxx
```

问题4：数据库操作异常

```
-- 现象：提示"数据库操作失败"

-- 排查步骤：
-- 1. 检查数据库连接
SHOW VARIABLES LIKE 'max_connections';

-- 2. 检查表是否存在
SHOW TABLES;

-- 3. 查看错误日志
SHOW VARIABLES LIKE 'log_error';

-- 4. 检查用户权限
SHOW GRANTS FOR 'root'@'localhost';
```

问题5：Redis缓存异常

```
# 现象：验证码无法验证/Token失效

# 排查步骤：
# 1. 检查Redis是否启动
redis-cli ping
# 预期返回：PONG

# 2. 查看Redis中的键
redis-cli KEYS "sms:*"
redis-cli KEYS "token:*"
redis-cli KEYS "login:fail:*"

# 3. 查看键的TTL
redis-cli TTL "sms:code:13800138000"

# 4. 删除指定键（测试用）
redis-cli DEL "sms:code:13800138000"
redis-cli DEL "login:fail:13800138000"
```

8.3 各模块问题排查

登录模块

错误现象	可能原因	解决方法
微信登录失败	appId/appSecret错误	检查application.yml中wx配置
账号密码错误	密码加密方式不一致	使用BCrypt加密密码
账号锁定	密码错误5次	等待15分钟或删除Redis键
短信发送失败	短信服务未配置	检查短信网关配置

商品模块

错误现象	可能原因	解决方法
图片上传失败	文件大小超限/格式错误	检查upload配置
商品审核失败	缺少必填字段	检查商品信息完整性
商品下架失败	状态不允许	检查商品当前状态

订单模块

错误现象	可能原因	解决方法
并发下单成功	分布式锁未生效	检查Redis连接
订单创建失败	商品已被下单	检查商品状态
取消订单失败	订单状态不允许	检查订单当前状态

IM模块

错误现象	可能原因	解决方法
消息发送失败	接收方不在线	检查在线状态
消息未送达	WebSocket断开	检查连接状态
未读数错误	Redis缓存不一致	刷新会话列表

8.4 调试工具使用

工具	用途	使用方法
微信开发者工具	小程序调试	Network/Console/AppData面板
Postman/Apifox	接口测试	导入接口集合
Navicat	数据库管理	连接MySQL查看数据
Redis Desktop Manager	Redis管理	连接Redis查看缓存
Chrome DevTools	管理端调试	Elements/Network/Console

8.5 日志查看位置

```
# 后端日志
# 控制台直接输出，或查看日志文件

# MySQL日志
# 位置：MySQL安装目录/data/*.log

# Redis日志
# 位置：Redis安装目录/redis.log

# Nginx日志（如使用）
# 位置：/var/log/nginx/*.log
```

✅ 联调检查签字

模块	联调人	完成日期	备注
登录注册			
首页			
搜索筛选			
商品模块			
收藏模块			
IM即时通讯			
订单模块			
评价模块			
消息中心			
管理端			

本指南用于指导真实接口联调执行